

- KNN 实验报告
 - 一、数据集要求
 - 1. 数据集选择
 - 2. 数据预处理
 - 二、算法实现要求
 - 1. KNN 模型构建
 - 2. 距离度量
 - 三、参数调优要求
 - 1. K 值选择
 - 2. 权重设置
 - 四、模型评估要求
 - 1. 评估指标
 - 2. 交叉验证
 - 五、可视化要求
 - 六、实验总结
 - 实验结果
 - 遇到的问题及解决
 - 主要发现
 - 改进方向
 - 实验步骤
 - 代码实现
 - 结果分析
 - 结论
 - 程序清单
- `X_train["Embarked"] = X_train["Embarked"].map() # 登船口数字化`
- `X_test["Embarked"] = X_test["Embarked"].map()`
- 定义要搜索的 K 和 p
- 交叉验证的评价指标
- 开始遍历
- 保存到表格
- =====
- 封装：画 KNN 指标折线图
- =====
 - 实验结果
 - 疑难小结

KNN 实验报告

一、数据集要求

1. 数据集选择

本次实验选用 Kaggle 平台公开的泰坦尼克号生还预测数据集，该数据集为经典二分类任务数据集，用于根据乘客信息预测其是否生还。数据集中包含乘客个人信息、家庭信息、船票信息以及最终是否生还的标签，其中训练集包含 891 条带标签样本，测试集包含 418 条无标签样本，用于最终预测。

特征含义如下：Pclass 代表乘客船舱等级，反映乘客社会地位；Sex 为乘客性别；Age 为乘客年龄；SibSp 为同行兄弟姐妹及配偶数量；Parch 为同行父母及子女数量；Fare 为乘客购买船票的价格，体现经济水平。标签为 Survived，取值 0 和 1，0 代表乘客死亡，1 代表乘客生还。

2. 数据预处理

实验对数据集进行了多项必要的预处理操作。首先对类别型特征进行数字化转换，将性别特征中的 male 映射为 0，female 映射为 1，使模型能够处理。其次处理缺失值，数据集中年龄、票价等特征存在缺失情况，采用中位数填充策略，使用中位数填充缺失数值，避免缺失值对模型训练的影响。同时为保证 KNN 模型正常运行，对所有特征进行标准化处理，将特征数据转换为均值为 0、方差为 1 的标准分布，消除特征量纲差异对距离计算的干扰，且所有预处理规则均仅从训练集学习，避免测试集信息泄露。

二、算法实现要求

1. KNN 模型构建

本次实验基于 scikit-learn 机器学习库中的 KNeighborsClassifier 类构建 KNN 分类模型，通过设置不同参数实现模型的初始化与训练。模型输入为经过预处理的乘客特征数据，输出为乘客是否生还的二分类结果，整体流程遵循训练、验证、预测的标准机器学习流程，未手动实现 KNN 核心逻辑，直接使用成熟库函数保证实验效率与稳定性。

2. 距离度量

实验对比了四种不同的距离度量方式，分别是曼哈顿距离、欧氏距离、闵可夫斯基距离和切比雪夫距离。通过设置不同参数调用对应距离计算方式，探究不同距离度量对 KNN 模型分类性能的影响，分析不同距离在该数据集上的适配程度。

三、参数调优要求

1. K 值选择

实验选取 1 至 15 的连续整数作为 K 值候选范围，遍历所有 K 值以寻找最优参数。通过观察不同 K 值下模型的各项性能指标发现，K 值过小时模型易受噪声影响出现过拟合，K 值过大时模型会过度泛化出现欠拟合，同时偶数 K 值易出现邻居投票平局的情况，导致模型偏向数据中的多数类，奇数 K 值则无平局问题，分类结果更稳定。

2. 权重设置

实验采用 KNN 模型默认的均匀权重设置，即所有近邻样本在投票时拥有相同的权重，未使用距离加权方式。该方式下每个邻居对最终分类结果的贡献完全相等，保证不同参数对比实验的公平性，可清晰观察 K 值与距离度量对模型的独立影响。

四、模型评估要求

1. 评估指标

实验选用准确率、精确率、召回率三项核心指标评估模型性能。准确率代表模型整体分类正确的样本占总样本的比例，反映模型整体表现；精确率代表模型预测为生还的样本中真实生还的比例，体现预测结果的可靠性；召回率代表真实生还的样本中被模型正确预测的比例，体现模型对正类样本的识别能力。

2. 交叉验证

实验采用 5 折交叉验证评估模型性能，将训练集平均划分为 5 份，轮流使用其中 4 份作为训练集、1 份作为验证集，循环 5 次后取各项指标的平均值作为最终结果。该方法无需使用测试集即可可靠评估模型泛化能力，避免单次数据划分带来的偶然性，为参数选择提供稳定依据。

五、可视化要求

实验绘制了不同 K 值与模型性能指标的关系曲线图，分别展示 K 值与准确率、K 值与精确率、K 值与召回率的变化趋势。每张图中包含不同距离度量对应的曲线，可直观对比不同参数组合下的模型性能，清晰呈现 K 值变化、距离度量差异对各项指标的影响规律。

六、实验总结

实验结果

在本次实验的参数范围内，曼哈顿距离整体表现优于其他距离度量，奇数 K 值对应的召回率显著高于偶数 K 值，模型整体准确率较高，但受数据集样本不平衡影响，召回率相对偏低，模型更倾向于将样本预测为死亡类别。

遇到的问题及解决

实验中遇到模型未训练就进行预测的错误，通过补充模型训练步骤解决；遇到中文绘图报错问题，通过使用英文图表内容规避；遇到交叉验证指标计算混乱问题，统一使用 5 折交叉验证规范计算各项指标；遇到环境变量与命令调用问题，通过正确配置环境与执行命令修复。

主要发现

K 值的奇偶性会通过投票机制显著影响召回率，偶数 K 易因平局偏向多数类；距离度量对 KNN 性能影响明显，不同数据集适配的距离方式不同；KNN 模型对特征尺度敏感，标准化是必要步骤；交叉验证是调参的可靠手段，测试集仅可用于最终预测。

改进方向

可尝试加入距离权重提升模型性能；补充特征工程挖掘更多有效信息；对比更多机器学习模型与 KNN 性能；调整缺失值填充策略与特征组合，进一步优化模型效果。

实验步骤

1. 导入所需Python库，读取训练集与测试集数据并保存测试集乘客编号；
2. 选取核心特征，对类别特征进行数字化转换；
3. 使用中位数填充特征缺失值；
4. 对特征数据进行标准化处理；
5. 遍历不同 K 值与距离度量，通过 5 折交叉验证计算各项评估指标；
6. 封装绘图函数，绘制性能指标随 K 值变化的曲线图；
7. 选取最优参数训练最终模型，对测试集进行预测；
8. 生成符合要求的预测结果文件。

代码实现

实验使用完整Python代码实现，代码包含数据读取、预处理、模型训练、参数遍历、交叉验证、可视化、结果输出等模块，添加详细注释，运行环境为配置好机器学习库的Python环境，可直接复现实验结果。

结果分析

模型整体分类准确率良好，不同距离度量中曼哈顿距离效果最优，K 值在较小奇数时综合性能最佳。偶数 K 值因投票平局导致召回率下降，体现了样本不平衡对 KNN 模型的影响。KNN 算法实现简单、可解释性强，但对特征尺度敏感、计算量大，在高维数据与大规模数据集上表现不佳。

结论

本次实验通过 KNN 算法完成泰坦尼克号生还预测，验证了 K 值、距离度量对模型性能的影响，掌握了数据预处理、交叉验证、模型评估与可视化的完整流程。KNN 算法适用于小规模、低维度的分类任务，在处理样本不平衡数据时需优化策略以提升正类识别能力，后续可通过优化参数、改进特征与调整模型结构进一步提升预测效果。

程序清单

```
import pandas as pd # 用于读取csv、处理表格数据 import numpy as np # 数值计算
from sklearn.neighbors import KNeighborsClassifier # KNN分类器 from
sklearn.preprocessing import StandardScaler # 特征标准化（KNN必须） from
sklearn.impute import SimpleImputer # sklearn官方缺失值填充器 from
sklearn.model_selection import train_test_split # 划分训练验证集 from sklearn.metrics
import accuracy_score # 计算准确率 import joblib from sklearn.model_selection import
cross_val_score
```

```
train = pd.read_csv("train.csv") # 训练集：含特征 + 是否生还标签 test =
pd.read_csv("test.csv") # 测试集：只有特征，没有标签
```

```
test_ids = test["PassengerId"] # 保存测试集乘客编号（最后提交必须）
```

```
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare"]
```

```
X_train = train[features].copy() # 训练集输入特征 X_test = test[features].copy() # 测试集输
入特征 y_train = train["Survived"].copy() # 训练集标签：0死亡 / 1生还
```

```
X_train["Sex"] = X_train["Sex"].map({"male": 0, "female": 1}) # 性别数字化 X_test["Sex"] =
X_test["Sex"].map({"male": 0, "female": 1})
```

```
X_train["Embarked"] =  
X_train["Embarked"].map() # 登船口数字  
化
```

```
X_test["Embarked"] =  
X_test["Embarked"].map()
```

```
imputer = SimpleImputer(strategy="median")
```

```
X_train_clean = imputer.fit_transform(X_train) # 拟合训练集并填充空值 X_test_clean =
imputer.transform(X_test) # 用训练集规则填充测试集（防止数据泄露）
```

```
scaler = StandardScaler() # 标准化器：均值0，方差1 X_train_scaled =
scaler.fit_transform(X_train_clean) # 训练集标准化 X_test_scaled =
scaler.transform(X_test_clean) # 测试集按同样规则标准化
```

```
from sklearn.metrics import make_scorer, accuracy_score, precision_score, recall_score, f1_score
```

定义要搜索的 K 和 p

```
k_list = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] p_list = [1,2,3,0]
```

交叉验证的评价指标

```
scoring = { 'accuracy': make_scorer(accuracy_score), 'precision':  
make_scorer(precision_score), 'recall': make_scorer(recall_score), 'f1':  
make_scorer(f1_score) }
```

开始遍历

```
results = [] for k in k_list: for p in p_list:
```

```
    if(p!=0) :  
  
        knn = KNeighborsClassifier(n_neighbors=k, p=p)  
        else :  
            knn = KNeighborsClassifier(n_neighbors=k, metric='chebyshev')  
  
        # 交叉验证（注意：必须用标准化数据！）  
        from sklearn.model_selection import cross_validate  
        acc = cross_val_score(knn, X_train_scaled, y_train, cv=5).mean()  
        acc = cross_val_score(knn, X_train_scaled, y_train, cv=5,  
scoring="accuracy").mean()  
        precision = cross_val_score(knn, X_train_scaled, y_train, cv=5,  
scoring="precision").mean()  
        recall = cross_val_score(knn, X_train_scaled, y_train, cv=5,  
scoring="recall").mean()  
  
        results.append([k, p, acc, precision, recall])
```

保存到表格

```
df = pd.DataFrame(results, columns=["K", "distance", "accuracy", "precision", "recall"])
```

```
import matplotlib.pyplot as plt
```

封装：画 KNN 指标折线图

```
def plot_knn_metric(df, k_list, metric_name, title, ylabel, save_path): plt.figure(figsize=(10, 5))
```

```
# 画每条线
for name in df["distance"].unique():
    sub = df[df["distance"] == name]
    plt.plot(sub["K"], sub[metric_name], marker='o', linewidth=2,
label=name)

# 样式
plt.title(title, fontsize=14)
plt.xlabel("K Value", fontsize=12)
plt.ylabel(ylabel, fontsize=12)
plt.xticks(k_list)
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

```
plot_knn_metric(df, k_list, "accuracy", "KNN Accuracy vs K", "Accuracy",
"knn_accuracy.png") plot_knn_metric(df, k_list, "precision", "KNN Precision vs K",
"Precision", "knn_precision.png") plot_knn_metric(df, k_list, "recall", "KNN Recall vs K",
"Recall", "knn_recall.png")
```

```
model = KNeighborsClassifier(n_neighbors=3, p=1) model.fit(X_train_scaled, y_train)
```

```
y_pred = model.predict(X_test_scaled)
```

```
submission = pd.DataFrame({ "PassengerId": test["PassengerId"], "Survived": y_pred, })
```

```
submission.to_csv("titanic_knn_result.csv", index=False)
```


实验结果

实验得到不同参数下模型的准确率、精确率、召回率数值，绘制出三张性能变化曲线图，最终生成测试集乘客的生还预测文件，确定了适用于该数据集的最优 K 值与距离度量组合。

疑难小结

实验中遇到模型未拟合、文件找不到、命令不可用、绘图报错等问题，通过检查代码逻辑、补充必要步骤、规范环境配置逐一解决。深刻认识到数据预处理对 KNN 的重要性、交叉验证在调参中的作用，以及参数合理性对模型性能的关键影响，提升了机器学习实验的实操与问题排查能力。